

Esame Programmazione di Sistema / API Programming

Data: 2023-06-20

QUESTIONS

Domanda 1

Si definisca il problema delle referenze cicliche nell'uso degli smart pointers. Si fornisca quindi un esempio in cui tale problema sia presente.

Domanda 2

Si identifichino i tratti fondamentali della concorrenza. Successivamente, in riferimento alla mutabilità/immutabilità delle risorse, si delinei come questa affligga la gestione della sincronizzazione a livello thread.

Domanda 3

Data la struttura dati definita come:

```
struct Data {  
    Element: AsVector,  
    next: Rc<Data>  
}  
  
enum AsVector {  
    AsVector(Box<Rc<i32>>),  
    None  
}
```

Indicare l'occupazione di memoria di un singolo elemento in termini di:

1. numero di byte complessivi (caso peggiore, architettura a 64 bit);
2. posizionamento dei vari byte in memoria (stack, heap, ecc.).

Domanda 4

La `struct MpMcChannel<E: Send>` è una implementazione di un canale su cui possono scrivere molti produttori e da cui possono attingere valori molti consumatori. Tale struttura offre i seguenti metodi:

- `new(n: usize) -> Self`
Crea una istanza del canale basato su un buffer circolare di `n` elementi.
- `send(e: E) -> Option<()>`
Invia l'elemento `e` sul canale. Se il buffer circolare è pieno, attende senza consumare CPU che si crei almeno un posto libero in cui depositare il valore. Ritorna:
 - `Some(())` se è stato possibile inserire il valore nel buffer circolare.
 - `None` se il canale è stato chiuso (Attenzione: la chiusura può avvenire anche mentre si è in attesa che si liberi spazio) o se si è verificato un errore interno.
- `recv() -> Option<E>`
Legge il prossimo elemento presente sul canale. Se il buffer circolare è vuoto, attende senza consumare CPU che venga depositato almeno un valore. Ritorna:
 - `Some(e)` se è stato possibile prelevare un valore dal buffer.
 - `None` se il canale è stato chiuso (Attenzione: se, all'atto della chiusura sono già presenti valori nel buffer, questi devono essere ritornati, prima di indicare che il buffer è stato chiuso; se la chiusura avviene mentre si è in attesa di un valore, l'attesa si sblocca e viene ritornato `None`) o se si è verificato un errore interno.
- `shutdown() -> Option<()>`
Chiude il canale, impedendo ulteriori invii di valori. Ritorna:
 - `Some(())` per indicare la corretta chiusura.
 - `None` in caso di errore interno all'implementazione del metodo.

Si implementi tale struttura dati in linguaggio Rust, senza utilizzare i canali forniti dalla libreria standard né da altre librerie, avendo cura di garantirne la correttezza in presenza di più thread e di non generare la condizione di panico all'interno dei suoi metodi.

SOLUTIONS

Soluzione 1

Il problema delle referenze cicliche si presenta quando due o più valori gestiti da smart pointer dotati di conteggio dei riferimenti (come `Rc<T>` o `Arc<T>`) si puntano a vicenda, creando un ciclo. Dato che il conteggio dei riferimenti (`strong_count`) di ogni elemento nel ciclo non scenderà mai a zero, la memoria allocata nello heap per tali elementi non verrà mai liberata, causando un memory leak. Per risolvere questo problema, Rust introduce i riferimenti deboli (`Weak<T>`), i quali permettono di puntare al dato senza incrementare lo `strong_count`. Un riferimento debole non garantisce la validità del dato e deve essere promosso a forte (tramite `upgrade()`) prima dell'uso.

Esempio di referenza ciclica (e potenziale memory leak senza Weak):

```
use std::rc::Rc;
use std::cell::RefCell;

struct Node {
    value: i32,
    next: Option<Rc<RefCell<Node>>>,
}

fn main() {
    let a = Rc::new(RefCell::new(Node {
        value: 1,
        next: None,
    }));

    let b = Rc::new(RefCell::new(Node {
        value: 2,
        next: Some(Rc::clone(&a)),
    }));

    // Creazione del ciclo: a punta a b, e b punta ad a
    a.borrow_mut().next = Some(Rc::clone(&b));

    // Al termine dello scope, il drop di a e b decrementa i loro
    // conteggi
    // a 1, ma mai a 0. I due nodi non verranno deallocati.
}
```

Soluzione 2

In Rust, i tratti fondamentali della concorrenza sono `Send` e `Sync`.

- **Send**: indica che il possesso (ownership) di un valore di quel tipo può essere trasferito in modo sicuro da un thread a un altro.
- **Sync**: indica che è sicuro condividere referenze immutabili (`&T`) a un valore di quel tipo tra più thread. Un tipo `T` è `Sync` se e solo se `&T` è `Send`.

Mutabilità e sincronizzazione: Rust fa rispettare la regola che si possono avere o più referenze immutabili, o una singola referenza mutabile. Per condividere un dato mutabile tra più thread, è necessario che il dato sia sia condiviso (tramite ad es. `Arc`) sia protetto tramite un meccanismo di sincronizzazione (interior mutability thread-safe) come `Mutex` o `RwLock`. Questi tipi espongono metodi che, attraverso sincronizzazione a livello thread, assicurano mutua esclusione e garantiscono l'ottenimento di un riferimento mutabile sicuro, rispettando le garanzie imposte dal borrow checker.

Soluzione 3

Assumendo architettura a 64 bit (i puntatori occupano 8 byte):

- `Rc<i32>` allocato sullo heap: La struttura interna di un `Rc` (`RcBox`) contiene i due contatori (`strong` e `weak`, ciascuno tipicamente di dimensione pari a un `usize`, ovvero 8 byte l'uno, totale 16 byte) più il dato `i32` (4 byte). A causa dell'allineamento a 8 byte, il blocco di memoria complessivo occupato sarà $8 + 8 + 4 = 20$ byte, arrotondato all'allineamento richiesto di 8 byte per un totale di **24 byte**.
- `Box<Rc<i32>`: un puntatore sullo stack (8 byte) che punta a un `Rc<i32>` (ovvero un puntatore a sua volta) sullo heap (altri 8 byte).
- `AsVector` enum: contiene il blocco `AsVector(Box<Rc<i32>)` oppure `None`. Rust ottimizza i tag delle `Option` e degli `enum` che contengono puntatori validi (non-nulli, come `Box`): il valore `None` è rappresentato dal puntatore nullo. Perciò, l'enum occuperà **8 byte** (dimensione del `Box`).
- `Rc<Data>`: il puntatore in sé occupa **8 byte**.
- `Data` struct: Composta da `Element` (8 byte) e `next` (8 byte). Totale **16 byte**.

1. Numero di byte complessivi: Per la struct `Data` allocata sullo stack: 16 byte (la dimensione della struct `Data`). Spazio totale sullo heap, se consideriamo il solo `Box` e il solo `Rc<i32>`, sarà: l'allocazione del `Box` (8 byte, contenente il puntatore del `Rc`) e l'allocazione interna del `Rc<i32>` (24 byte). In totale, per un singolo elemento `Data`: **16 byte sullo stack e fino a 32 byte sullo heap**.

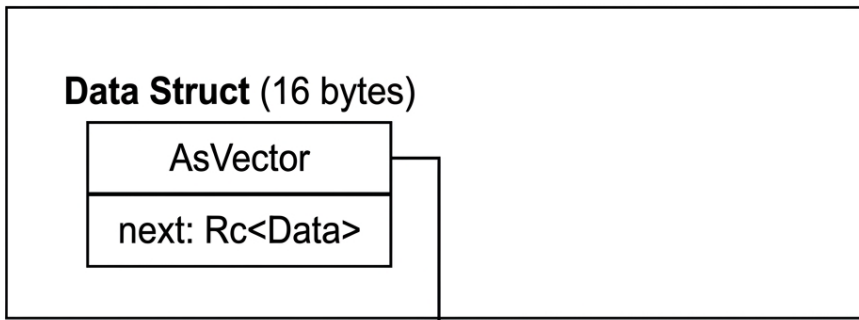
2. Posizionamento dei vari byte:

- **Nello Stack:** I **16 byte** della struct `Data` (8 byte per il puntatore `Box` in `AsVector`, 8 byte per il puntatore `Rc` in `next`).
- **Nello Heap:** Se `AsVector` è popolato, c'è un blocco di **8 byte** per il contenuto del `Box` (il puntatore `Rc<i32>`), e l'area di memoria puntata dal `Rc` che ospita contatori (16 byte) e il valore `i32` (con padding, 8 byte) per un totale di **24 byte**. Totale heap: 32 byte.

Rappresentazione Grafica

L'immagine seguente illustra in modo preciso la disposizione della memoria analizzata. I blocchi evidenziano come i puntatori presenti sullo Stack (`AsVector`) si colleghino agli indirizzi allocati sullo Heap (il blocco del `Box`), il quale a sua volta inoltra la dereferenziazione al control block dell'`Rc`.

STACK



HEAP

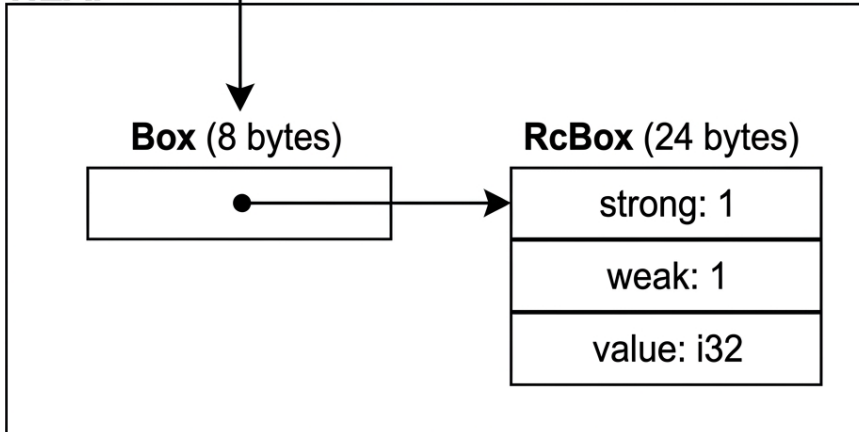


Figura 1: Rappresentazione Architetture dello Stack e dell'Heap

Soluzione 4

```
use std::collections::VecDeque;
use std::sync::{Condvar, Mutex};

#[derive(PartialEq)]
enum ChannelState {
    Open,
    Closed,
}

pub struct MpMcChannel<E: Send> {
    state_buffer: Mutex<(ChannelState, VecDeque<E>>),
    buffer_size: usize,
    cv_send: Condvar,
    cv_recv: Condvar,
}

impl<E: Send> MpMcChannel<E> {
    pub fn new(n: usize) -> Self {
        MpMcChannel {
```

```

        state_buffer: Mutex::new((ChannelState::Open, VecDeque
            ::with_capacity(n))),
        buffer_size: n,
        cv_send: Condvar::new(),
        cv_recv: Condvar::new(),
    }
}

pub fn send(&self, e: E) -> Option<()> {
    let mut guard = self.state_buffer.lock().ok()?;

    while guard.1.len() == self.buffer_size && guard.0 ==
        ChannelState::Open {
        guard = self.cv_send.wait(guard).ok()?;
    }

    if guard.0 == ChannelState::Closed {
        return None;
    }

    guard.1.push_back(e);
    self.cv_recv.notify_one();
    Some(())
}

pub fn recv(&self) -> Option<E> {
    let mut guard = self.state_buffer.lock().ok()?;

    while guard.1.is_empty() && guard.0 == ChannelState::Open
    {
        guard = self.cv_recv.wait(guard).ok()?;
    }

    if guard.1.is_empty() && guard.0 == ChannelState::Closed {
        return None;
    }

    let val = guard.1.pop_front()?;
    self.cv_send.notify_one();
    Some(val)
}

pub fn shutdown(&self) -> Option<()> {
    let mut guard = self.state_buffer.lock().ok()?;
    guard.0 = ChannelState::Closed;
    // Sveglia sia senders che receivers in attesa per
    sbloccarli e far ritornare None
    self.cv_send.notify_all();
    self.cv_recv.notify_all();
    Some(())
}

```

}